

Practical Task 2 – Research Questions

Django – eCommerce Application

1. Python Requests Library

The Python requests library is used to make HTTP requests and communicate with web servers. It has an intuitive and simple structure, which makes it easier to use compared to some other libraries. Some of its common uses include testing websites, web scraping, and working with APIs.

When using an external library, the first step is to install it. We can install the requests library using the following command:

```
pip install requests
```

On some macOS or Linux systems, we can also use:

```
pip3 install requests
```

After installing the library, we need to import it into our Python program:

```
import requests
```

The requests library supports different HTTP methods, such as GET, POST, PUT, and DELETE. In this example, I am using a GET request. A GET request is used to retrieve information from a server through a URL.

For this example, I used the following URL:

```
url = "https://jsonplaceholder.typicode.com/posts/1"  
response = requests.get(url)
```

The program sends a GET request to the URL and stores the server's response in the response variable. Then, we can check the status code and the type of content returned by the server:

```
print("Status code: ", response.status_code)  
print("Content-Type: ", response.headers.get('Content-Type'))
```

The response was:

```
Status code: 200  
Content-Type: application/json; charset=utf-8
```

The status code 200 means that the request was successful. The Content-Type shows that the server returned the information in JSON format.

Finally, we can use the `.json()` method to convert the JSON response into Python data that we can work with:

```
data = response.json()
print(data)
```

The result was:

```
{'userId': 1, 'id': 1, 'title': 'sunt aut facere repellat provident occaecati
excepturi optio reprehenderit', 'body': 'quia et suscipit\nsuscipit recusandae
consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum
rerum est autem sunt rem eveniet architecto'}
```

This example shows how the requests library can be used to communicate with a web server, retrieve data using a GET request, and process the returned JSON data in Python.

2. JSON and XML Data Formats

XML

XML (Extensible Markup Language) is a markup language used to represent, store, and exchange structured data between different systems and applications. It uses tags to describe the information and organize it in a structured way.

For example:

```
<person>
<name>Jose</name>
<age>27</age>
</person>
```

XML uses an opening tag, such as <name>, and a closing tag, such as </name>. The tags describe what each piece of information represents.

Advantages of XML

1. Structured data: XML provides a clear structure for organizing complex data.
2. Interoperability: XML can be used between different programming languages and operating systems.
3. Scalability: It can handle large and complex amounts of structured data.
4. Schema support: XML can use schemas to define and validate the structure and rules of the data.

Disadvantages of XML

1. More verbose: XML usually requires more text because every element needs opening and closing tags.
2. Larger file size: The extra tags can make XML files larger than equivalent JSON files.
3. More processing: XML requires parsing, which can require more memory and processing than simpler formats.

4. More complex to work with: XML can be harder to read and manage when the data structure becomes very large or complicated.

What is XML used for?

XML is commonly used in enterprise and legacy systems, document markup, SOAP web services, APIs, and configuration files where a strict structure or schema validation is required.

JSON

JSON (JavaScript Object Notation) is a lightweight data format used to represent and exchange structured data. It is commonly used for communication between web applications, servers, APIs, and different software systems.

JSON uses key-value pairs and structures such as objects and arrays. Objects are enclosed in curly braces {}, while arrays are enclosed in square brackets [].

For example:

```
{
  "name": "Jose",
  "age": 27,
  "skills": ["Python", "SQL"]
}
```

JSON supports different types of values, including strings, numbers, booleans, arrays, objects, and null. Its syntax is also similar to the way data is represented in many modern programming languages, which makes it relatively easy for developers to read and use.

Advantages of JSON

1. Lightweight: JSON usually requires less data than XML, making it efficient for transferring information.
2. Easy to read: Its simple syntax makes JSON easy for both humans and computers to understand.
3. Easy to use with programming languages: Most modern programming languages have built-in or external tools for working with JSON.
4. Good for web applications: JSON is widely supported by web browsers, APIs, and web services.

Disadvantages of JSON

1. Less suitable for complex documents: JSON can become difficult to manage when the data structure is very complex.
2. Limited data types: JSON has fewer built-in data types compared to some other data formats.
3. Less strict validation: JSON does not have the same built-in schema and validation capabilities that XML can provide.
4. JSON doesn't support comments.

What is JSON used for?

JSON is commonly used in modern web APIs, front-end applications, client-server communication, mobile applications, and back-end services. It is especially popular for exchanging data between a client and a server.

3. RESTful API

A RESTful API is an interface that allows two different systems or applications to communicate and exchange data. A good example is a business application that needs to communicate with a bank's system to make a payment. The business application can use an API to send the required information to the bank and receive a response. Security can be added using HTTPS, authentication, and authorization.

REST stands for Representational State Transfer. It is an architectural style that defines principles for designing web services. APIs that follow these principles are called RESTful APIs.

How does a RESTful API work?

A RESTful API works through communication between a client and a server. The client makes a request when it needs information or wants to perform an action, and the server processes the request and sends a response.

The process usually works like this:

1. The client sends a request to the server using the API.
2. The client follows the API documentation to format the request correctly.
3. The server authenticates the client and checks whether it has permission to perform the requested action.
4. The server receives and processes the request.
5. The server sends a response back to the client.

A RESTful request can contain several important components:

- URL: Identifies the resource that the client wants to access. For example, a URL can identify a specific user, product, or order.
- HTTP method: Indicates what the client wants to do with the resource. The most common methods are GET, POST, PUT, and DELETE.
- Headers: Provide additional information about the request and response, such as the content type or authentication information.
- Parameters: Provide additional information to the server, such as filters, page numbers, or search values.
- Body: Contains data that the client sends to the server, which is commonly used with methods such as POST and PUT.

For example, a GET request can ask the server for information about a specific user. The server processes the request and can return the information in JSON format.

Advantages of RESTful APIs

1. Simple and easy to understand: REST uses standard HTTP methods, making it relatively easy for developers to work with.
2. Scalable: RESTful APIs can support many clients and large amounts of requests.
3. Flexible: They can communicate with different programming languages, platforms, and applications.
4. Stateless: Each request contains the information needed to process it, so the server does not need to remember the client's previous requests.

Disadvantages of RESTful APIs

1. Security is not automatic: REST itself does not provide security. Developers need to implement HTTPS, authentication, authorization, and other security measures.
2. Multiple requests may be required: A client may need to make several requests to obtain all the information it needs.
3. Over-fetching or under-fetching: An API may return more information than the client needs or not provide enough information in a single request.
4. Not ideal for every application: REST may not be the best option for systems that require real-time communication or very complex data relationships.

What are RESTful APIs used for?

RESTful APIs are widely used to connect different applications and services. They are commonly used in websites, mobile applications, e-commerce systems, banking applications, and other systems that need to exchange data between a client and a server.